

TASK MANAGEMENT SYSTEM

Distributed Java RMI Application

Mukanire Jospin Lukogo

Project Documentation

Advanced Object-Oriented Programming / Distributed Systems

Bachelor of Business Information Technology

Table of Contents

1. Project Overview	3
1.1 Objectives	3
2. System Architecture	3
2.1 High-Level Component Diagram	3
2.2 Client Tier	3
2.3 Server Tier	3
2.4 Data Tier	4
3. Technology Stack	4
4. Package Structure	4
5. Service Layer (RMI)	4
5.1 Representative Sub-Interface Groups	4
5.2 Client-Side Access Pattern	5
6. Data Model	5
6.1 Employee	5
6.2 Task	6
7. Client Application (JavaFX GUI)	6
7.1 Login	6
7.2 Registration	6
7.3 Employee Dashboard	7
7.4 Manager Dashboard	7
8. Database Design (PostgreSQL)	7
8.1 Notable Fixes Applied to the Data Layer	7
9. Testing Strategy	7
10. Build & Packaging	8
10.1 Maven	8
10.2 Native Installer via jpackage	8
11. Notable Issues Resolved During Development	8
12. Future Enhancements	8

1. Project Overview

The Task Management System is a distributed desktop application that allows the user to manage employees and the tasks assigned to them. It follows a classic client / RMI-server architecture: a JavaFX desktop client provides the user interface, while all business logic and data access run on a central Java RMI server backed by a PostgreSQL database.

The system supports two roles — Manager and Employee — each with a dedicated dashboard. Managers can create employee accounts, assign and cancel tasks, and monitor overall progress across the organization. Employees can log in, view the tasks assigned to them, and mark tasks as complete.

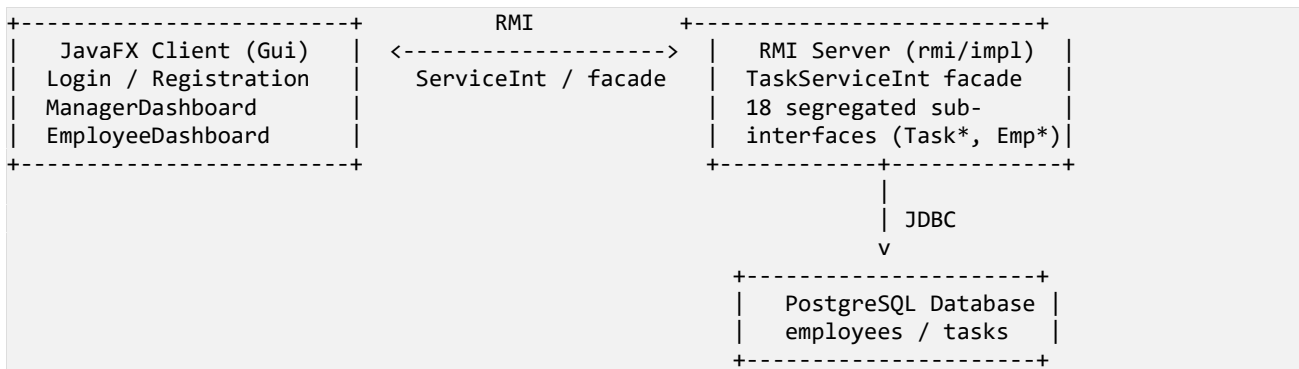
1.1 Objectives

- Demonstrate a working distributed system built with Java RMI, moving beyond a monolithic single-tier design.
- Apply the Interface Segregation Principle by decomposing a large service contract into focused sub-interfaces aggregated behind one facade.
- Provide a persistent, relational backing store (PostgreSQL) accessed exclusively through the server tier.
- Deliver a usable JavaFX client with role-based dashboards, packaged as a native installer for distribution.

2. System Architecture

The application is split into two independently runnable tiers that communicate over Java RMI (Remote Method Invocation). The client never talks to the database directly — every read or write goes through the remote service interface, which keeps data access logic, validation, and transaction handling centralized on the server.

2.1 High-Level Component Diagram



2.2 Client Tier

Built with JavaFX. Each screen (Login, Registration, ManagerDashboard, EmployeeDashboard) is its own Application subclass, constructed programmatically (no FXML). The client obtains a single shared remote stub via a `ClientMain.getService()` helper, which performs the RMI lookup once and hands back a reusable `ServiceInt` / `TaskServiceInt` reference.

2.3 Server Tier

The server exposes a single remote facade, referred to as `TaskServiceInt` (surfaced to the client code as `ServiceInt`), which aggregates roughly eighteen narrower sub-interfaces. Each sub-interface groups a related set of operations (for example, task CRUD, task status/statistics queries, employee CRUD, and authentication) so that no single interface grows into a god-interface, while the client still only needs to look up and hold one remote object.

2.4 Data Tier

PostgreSQL stores two core tables mirrored by the domain model: employees and tasks. All SQL access is confined to DAO-style classes on the server; the client has no JDBC dependency at all.

3. Technology Stack

Layer	Technology
Client UI	JavaFX (programmatic scenes, CSS-style inline styling)
Remote Communication	Java RMI (<code>java.rmi.Remote</code> / <code>UnicastRemoteObject</code>)
Server / Business Logic	Java (facade + segregated service interfaces)
Persistence	PostgreSQL, accessed via JDBC / DAO classes
Build Tool	Maven
Testing	JUnit 5, TestNG, Mockito
Packaging / Distribution	jpackage (native installer)
IDE	NetBeans

4. Package Structure

```

project-root/
  src/main/java/
    Gui/          - JavaFX screens (Login, Registration,
                  ManagerDashboard, EmployeeDashboard)
    model/        - Domain classes (Employee, Task) - Serializable
    rmi/          - ServiceInt / TaskServiceInt facade, sub-interfaces,
                  ClientMain (client-side lookup helper),
                  server-side implementation & DAO classes
  src/test/java/  - JUnit 5 / TestNG / Mockito test suites
  pom.xml        - Maven build configuration

```

5. Service Layer (RMI)

The remote contract is designed around the Interface Segregation Principle: rather than one large interface with every method a client could ever need, the system defines many small, cohesive sub-interfaces and combines them into a single facade, `TaskServiceInt` (referenced in client code as `ServiceInt`), which the server implementation implements in full and the client looks up as one object.

5.1 Representative Sub-Interface Groups

Group	Responsibility	Example Methods
-------	----------------	-----------------

Group	Responsibility	Example Methods
Authentication	Credential validation and account status	checklogin(email, password)
Employee CRUD	Create, read, update employee records	insertoperation(Employee), getAllEmployees()
Employee Stats	Aggregate counts for dashboards	countAllEmployees()
Task CRUD	Create and list tasks	inserttask(Task), getAllTasks()
Task Assignment / Query	Per-employee task retrieval	sortTasksByEmployee(employeeId)
Task Status	Update / transition task state	updatepro(taskId, status)
Task Statistics	Dashboard counters	countAllTasks(), countByStatus(status), countBytaskEmployee(id), countBystat(id, status)

This is a representative slice of the full contract; the complete facade spans about eighteen sub-interfaces in total, split further by concern (e.g. separate read vs. write groupings) than shown above.

5.2 Client-Side Access Pattern

Every screen follows the same pattern when it needs server data:

```
ServiceInt service = ClientMain.getService();
Employee employee = service.checklogin(email, password);
// ... use the returned object to drive UI state
```

Every remote call is wrapped in a try/catch block on the client; failures (connection issues, validation errors surfaced as exceptions) are caught and shown to the user via a status Label rather than crashing the UI thread.

6. Data Model

6.1 Employee

Field	Type	Notes
employeeId	int	Primary key, generated by the database
fullName	String	Display name shown across all dashboards
email	String	Used as the login identifier; must contain “@”
age	int	Must be 18+ per the registration form
password	String	Entered via PasswordField; validated server-side
role	String	“MANAGER” or “EMPLOYEE”

Field	Type	Notes
active	boolean	Inactive accounts are blocked at login
createdAt	LocalDateTime	Registration timestamp

6.2 Task

Field	Type	Notes
taskId	int	Primary key, generated by the database
title	String	Short task name
description	String	Free-text details
assignedTo	int	Foreign key → employeeId
status	String	PENDING / In progress / Done / Cancelled
dueDate	LocalDate	Parsed from a “YYYY-MM-DD” text field
createdAt	LocalDateTime	Set on insert
updatedAt	LocalDateTime	Refreshed on status change

Both classes implement `Serializable` so that instances can be passed as RMI method arguments and return values.

7. Client Application (JavaFX GUI)

Every screen is a standalone `javafx.application.Application` subclass so it can also be launched independently for testing. Styling is applied via inline `setStyle()` calls; recent revisions gave every screen a consistent white-card-on-light-grey visual language, rounded corners, soft drop shadows, and a shared purple/green/blue accent palette drawn from the role badges.

7.1 Login

Collects an email and password, calls `checklogin()` on the remote service, and routes the user based on the returned `Employee`:

- Invalid credentials or a null result → an error message is shown, no navigation occurs.
- Inactive account → blocked with a message directing the user to contact their manager.
- Role = `MANAGER` → opens `ManagerDashboard` and closes the login window.
- Role = `EMPLOYEE` → opens `EmployeeDashboard` and closes the login window.

A “Create Account” link opens the Registration screen.

7.2 Registration

Captures full name, email, age, password, role, and an optional “active” flag. Client-side validation runs before any remote call:

- All fields are required.
- Email must contain “@”.
- Selecting role MANAGER reveals a manager-code field, which must match a fixed code before submission succeeds.

On success, `insertoperation()` is called with a new Employee, the form is cleared, and the user is returned to the Login screen.

7.3 Employee Dashboard

On load, the dashboard fetches the logged-in employee’s task count, pending count, completed count, and full task list in a single try/catch block, then renders:

- Three summary tiles — My tasks, Pending, Completed.
- A card per task showing title, description, due date, task ID, assignment date, and a colour-coded status badge.
- A “Mark as Done” button per task (disabled once a task is Done or Cancelled) that calls `updatepro(taskId, “Done”)` and updates the badge in place without a full page reload.
- A Logout button that returns to the Login screen.

7.4 Manager Dashboard

The richest screen in the client. On load it fetches organization-wide totals (total tasks, in-progress count, employee count) and renders:

- Three summary tiles — Total Task, In Progress, Employees.
- An action bar with “Assign Task” and “Cancel Task” buttons.
- A scrollable list of all tasks in the system with assignee and due date.
- A collapsible “New Task” form (revealed by “Assign Task”) with title, description, an employee picker populated from `getAllEmployees()`, and a due-date field; submission builds a Task and calls `inserttask()`.
- A “Cancel a task” panel that takes a task ID and calls `updatepro(id, “Cancelled”)`.

8. Database Design (PostgreSQL)

The relational schema mirrors the domain model described in Section 6: an employees table and a tasks table, with `tasks.assigned_to` as a foreign key into `employees`. All access happens through DAO classes on the server; connections are opened and released in try/finally blocks to avoid leaks under repeated dashboard refreshes.

8.1 Notable Fixes Applied to the Data Layer

- Foreign-key violations caused by unassigned constructor fields when inserting new Task/Employee rows were resolved by ensuring every constructor parameter is populated before insert.
- A date-parsing bug affecting single-digit days (e.g. 2025-6-1 instead of 2025-06-01) was fixed by enforcing the zero-padded “YYYY-MM-DD” format expected by `LocalDate.parse()`.
- Column name typos between DAO SQL and the actual schema were corrected after being traced through failing insert/update calls.

9. Testing Strategy

Server-side logic is covered with a mix of testing frameworks chosen per concern:

Framework	Used For
JUnit 5	Core unit tests for service and DAO logic
TestNG	Additional suite-style and data-driven test scenarios
Mockito	Mocking the database layer and remote dependencies so service logic can be tested in isolation

Tests focus on the service layer's validation rules (e.g. login checks, status transitions) and on DAO methods against a controlled or mocked data source, rather than exercising the JavaFX UI directly.

10. Build & Packaging

10.1 Maven

The project is built with Maven, which manages JavaFX, JDBC driver, and testing dependencies (JUnit 5, TestNG, Mockito) and produces the runnable artifact consumed by the packaging step.

10.2 Native Installer via jpackage

The client is packaged into a native installer using jpackage, bundling a minimal Java runtime so end users do not need a separate JDK/JavaFX installation to run the application.

11. Notable Issues Resolved During Development

Issue	Resolution
StackOverflowError during startup/navigation	Removed recursive re-invocation between screens; screens now open a fresh Stage instead of re-entering the same call chain.
Foreign-key violations on insert	Ensured all required constructor fields are populated before calling insertoperation()/inserttask().
Single-digit day date parsing failure	Enforced zero-padded YYYY-MM-DD input before LocalDate.parse().
Column name typos in DAO SQL	Reconciled DAO field references against actual PostgreSQL column names.
Compiler error: cannot find symbol rgb(int,int,int,double)	A local String variable named Color (used for status badge colors) shadowed the javafx.scene.paint.Color class inside the same loop where a DropShadow color was being built. Fixed by hoisting the shadow color into a class-level CARD_SHADOW_COLOR constant declared outside the loop's scope.

12. Future Enhancements

- Move hard-coded manager registration code to a server-side configurable value or an admin-managed invite flow.

- Add pagination or filtering to the Manager Dashboard's task list as data volume grows.
- Introduce connection pooling (e.g. HikariCP) on the server for more efficient PostgreSQL access under concurrent load.
- Expand automated test coverage to include integration tests against a real (test) PostgreSQL instance.
- Replace inline `setStyle()` calls with an external CSS stylesheet for easier theming and maintenance.